06 — Projection, Compression, and Rank Allocation

SEION-KGR v26 MAX

Campaign ID: gate12-closeout-2026-08-01
Canonical commit: `6af3c35271ae2ffab41ecba2aad098d1988fdc0c`
Branch: `campaign/gate12-closeout`
Date: 2026-08-01

Purpose: state what the projection layer and rank controller actually do, what is tested, and what a Pareto study would require.

Contents

1 Projection layer

`seion_kgr/kernels.py::StiefelProjector`: Q on the Stiefel manifold via QR retraction of a learned parameter, $P = QQ^*$. **Status: PROVED_UNDER_ASSUMPTIONS** that $Q^*Q = I$ holds by construction of QR (any R), verified after real gradient steps (isometry/idempotent residuals $\sim 10^{-7}$, not just at init) — this caught a real bug during the base engineering work: an earlier sign-disambiguation step (`torch.sign(...).clamp_min(1e-12)`) silently replaced negative signs with 10^{-12} and shrank that column to near zero, breaking orthonormality after training; fixed by removing the unneeded correction (QR's Q is orthonormal regardless of sign convention).

`seion_kgr/projection.py::measure_closure_leakage_sample` computes the sample-mean closure-leakage ratio — explicitly labeled an `empirical_error_predictor`, never a certified bound (§07).

2 Rank allocation policies

Six policies in `seion_kgr/rank_controller.py`: `uniform`, `singular_energy`, `gradient_sensitivity`, `pathwise`, `local_error_greedy`, `hybrid` (the recommended default, a weighted combination of the other four's signals), plus `random` as a control. **Status: DEFINITION**, all six unit-tested for budget/max-rank compliance; `hybrid` verified structurally distinct from `pathwise` alone (never defaults to the single signal the confirmatory `adaptive_tensor_network` evidence already showed loses to `uniform` and `local_error_greedy` at equal budget — assumption A12).

`compare_policies` runs every policy through a caller-supplied objective and reports regret against the best-tried (not a proven oracle) — unit-tested with a hand-computed objective, not run against a real trained model's rank-allocation decision in this campaign.

3 Instrumentation wired into training

`seion_kgr/train.py`'s `_module_diagnostics` computes, per epoch (when the relevant branch is enabled): closure leakage (path reasoner only, if its projector is enabled), singular-energy-uncaptured (via exact SVD of the CP law's output weight matrix), and gradient-sensitivity (last-batch gradient norm on that module's parameters, an explicit simplification stated in code comments). These feed `rank_history.jsonl` together with a `compare_policies` call each eval epoch — real, executed, verified end to end on real WN18RR data (`campaigns/gate12/audit_smoke_run_selector` class of runs during Phase B development; superseded by Tier 0 for this package's canonical evidence).

4 What is NOT done: a Pareto study

No MRR-vs-rank/VRAM/latency Pareto curve was generated on benchmark data in this campaign. The instrumentation to produce one exists and is tested (above), but running it across a rank sweep on FB15K-237/WN18RR at meaningful scale requires the same path-reasoner-scaling resolution needed for a real confirmatory campaign (§09) — not attempted here, given the session's compute budget was spent on engineering closure and a bounded 2-ladder-point screening pass instead, per the mandate's own priority order (item 7, "projection/rank-controller Pareto study", explicitly ranked below primary screening and confirmatory work).

Status: OPEN. This is the single most concrete, cheaply-scoped follow-up: given the instrumentation already exists, a rank sweep at fixed budget on the Tier 0 A3 configuration (TuckER + path + seion, subsampled data) would produce a real, if still screening-tier, Pareto curve within a session of similar length to this one.